

All About Cmdlets and Modules

In this section, we will cover the following:

- What are cmdlets and Modules?
- How do we interact with them?
- How do we install and load new modules from the web?

Understanding these questions is crucial when utilizing PowerShell as both a sysadmin and pentester. PowerShells' ability to be modular and expandable makes it a powerhouse tool to have in our kit. Let us dive into what cmdlets and modules are.

Cmdlets

A [cmdlet](#) as defined by Microsoft is:

"a single-feature command that manipulates objects in PowerShell."

Cmdlets follow a Verb-Noun structure which often makes it easier for us to understand what any given cmdlet does. With Test-WSMan, we can see the [verb](#) is [Test](#) and the [Noun](#) is [Wsman](#). The verb and noun are separated by a dash (-). After the verb and noun, we would use the options available to us with a given cmdlet to perform the desired action. Cmdlets are similar to functions used in PowerShell code or other programming languages but have one significant difference. Cmdlets are [not](#) written in PowerShell. They are written in C# or another language and then compiled for use. As we saw in the last section, we can use the [Get-Command](#) cmdlet to view the available applications, cmdlets, and functions, along with a trait labeled "CommandType" that can help us identify its type.

If we want to see the options and functionality available to us with a specific cmdlet, we can use the [Get-Help](#) cmdlet as well as the [Get-Member](#) cmdlet.

PowerShell Modules

A [PowerShell module](#) is structured PowerShell code that is made easy to use & share. As mentioned in the official Microsoft docs, a module can be made up of the following:

- Cmdlets
- Script files
- Functions
- Assemblies
- Related resources (manifests and help files)

Through this section, we are going to use the PowerView project to examine what makes up a module and how to interact with them. [PowerView.ps1](#) is part of a collection of PowerShell modules organized in a project called [PowerSploit](#) created by the [PowerShellMafia](#) to provide penetration testers with many valuable tools to use when testing Windows Domain/Active Directory environments. Though we may notice this project has been archived, many of the included tools are still relevant and useful in pen-testing today (written in August 2022). We will not extensively cover the usage and implementation of PowerSploit in this module. We will just be using it as a reference to understand PowerShell better. The use of PowerSploit to Enumerate & Attack Windows Domain environments is covered in great depth in the module [Active Directory Enumeration & Attacks](#).



AntivirusBypass	For ./Antivirus/ :	6 years ago
CodeExecution	For ./CodeExecution/ :	6 years ago
Exfiltration	+Version check fix	5 years ago
Mayhem	For ./Mayhem/ :	6 years ago
Persistence	Fix for profile cleanup	5 years ago
Privesc	Merge pull request #238 from hackjammer/dev	5 years ago
Recon	swapped default kerberoasting output formats	4 years ago
ScriptModification	For ./ScriptModification/ :	6 years ago
Tests	PowerUp update:	6 years ago
docs	For ./Recon/ :	6 years ago
.gitignore	Revert "Normalizing all files to ascii encoding"	7 years ago
LICENSE	Changed licensing to BSD 3-Clause	10 years ago
PowerSploit.psd1	Fix build for import and run of Invoke-PrivescAudit	5 years ago
PowerSploit.psm1	Fix build for import and run of Invoke-PrivescAudit	5 years ago
PowerSploit.pssproj	Updated .psproj to reflect additions/removals	7 years ago
PowerSploit.sln	Adding Visual Studio 2015 project file	7 years ago
README.md	Updated README.md	2 years ago
mkdocs.yml	For ./Recon/ :	6 years ago

README.md

This project is no longer supported

PowerSploit is a collection of Microsoft PowerShell modules that can be used to aid penetration testers during all phases of an assessment. PowerSploit is comprised of the following modules and scripts:

Readme

View license

9.8k stars

765 watching

4.1k forks

Releases 2

PowerSploit on Dec 18, 201

+ 1 release

Packages

No packages published

Contributors

+ 21 contributors

Languages

PowerShell 96%

Other 0.6%

PowerSploit.psd1

A PowerShell data file (`.psd1`) is a [Module manifest file](#). Contained in a manifest file we can often find:

- Reference to the module that will be processed
- Version numbers to keep track of major changes
- The GUID
- The Author of the module
- Copyright
- PowerShell compatibility information
- Modules & cmdlets included
- Metadata

PowerSploit.psd1

master PowerSploit / PowerSploit.psd1

James McGinnigle Fix build for import and run of Invoke-PrivescAudit Latest commit f9b95c

2 contributors

174 lines (158 sloc) | 5.15 KB

Raw

```
1 @{
2 # Script module or binary module file associated with this manifest.
3 ModuleToProcess = 'PowerSploit.psm1'
4
5 # Version number of this module.
6 ModuleVersion = '3.0.0.0'
7
8 # ID used to uniquely identify this module
9 GUID = '6753b496-d842-40a3-924a-0f09e248640c'
10
11 # Author of this module
12 Author = 'Matthew Graeber'
13
14 # Copyright statement for this module
15 Copyright = 'BSD 3-Clause'
16
17 # Description of the functionality provided by this module
18 Description = 'PowerSploit is a collection of Microsoft PowerShell modules that can be used to aid penetration testers and red team operator dur
```

PowerSploit.psm1

A PowerShell script module file (**.psm1**) is simply a script containing PowerShell code. Think of this as the meat of a module.

Contents of PowerSploit.psm1

All About Cmdlets and Modules

```
Get-ChildItem $PSScriptRoot | ? { $_.PSIsContainer -and !('Tests','docs' -contains $_.Name) } | % { Import-Module $_.FullName
```

The `Get-ChildItem` cmdlet gets the items in the current directory (represented by the `$PSScriptRoot` automatic variable), and the `Where-Object` cmdlet (aliased as the `"?"` character) filters those down to only the items that are folders and do not have the names "Tests" or "docs". Finally, the `ForEach-Object` cmdlet (aliased as the `"%"` character) executes the `Import-Module` cmdlet against each of those remaining items, passing the `DisableNameChecking` parameter to prevent errors if the module contains cmdlets or functions with the same names as cmdlets or functions in the current session.

Using PowerShell Modules

Once we decide what PowerShell module we want to use, we will have to determine how and from where we will run it. We also must consider if the chosen module and scripts are already on the host or if we need to get them on to the host. `Get-Module` can help us determine what modules are already loaded.

Get-Module

All About Cmdlets and Modules

```
PS C:\htb> Get-Module
```

ModuleType	Version	Name	ExportedCommands
Script	0.0	chocolateyProfile	{TabExpansion, Update-SessionEnvironment, refreshenv}
Manifest	3.1.0.0	Microsoft.PowerShell.Management	{Add-Computer, Add-Content, Checkpoint-Computer, Clear-
Manifest	3.1.0.0	Microsoft.PowerShell.Utility	{Add-Member, Add-Type, Clear-Variable, Compare-Object,
Script	0.7.3.1	posh-git	{Get-PoshGitToProfile, Add-SshKey, Enable-GitColors,
Script	2.0.0	PSReadline	{Add-PSReadLineKeyHandler, Get-PSReadLineOption, Remo

List-Available

All About Cmdlets and Modules

```
PS C:\htb> Get-Module -ListAvailable
```

```
Directory: C:\Users\tru7h\Documents\WindowsPowerShell\Modules
```

ModuleType	Version	Name	ExportedCommands
Script	1.1.0	PSSQLite	{Invoke-SqliteBulkCopy, Invoke-SqliteQuery, New-Sqli...

```
Directory: C:\Program Files\WindowsPowerShell\Modules
```

ModuleType	Version	Name	ExportedCommands
Script	1.0.1	Microsoft.PowerShell.Operation.V...	{Get-OperationValidation, Invoke-OperationValidation}
Binary	1.0.0.1	PackageManagement	{Find-Package, Get-Package, Get-PackageProvider, Get...
Script	3.4.0	Pester	{Describe, Context, It, Should...}
Script	1.0.0.1	PowerShellGet	{Install-Module, Find-Module, Save-Module, Update-Mod...
Script	2.0.0	PSReadLine	{Get-PSReadLineKeyHandler, Set-PSReadLineKeyHandler,

The `-ListAvailable` modifier will show us all modules we have installed but not loaded into our session.

We have already transferred the desired module or scripts onto a target Windows host. We will then need to run them. We can start through the use of the `Import-Module` cmdlet.

Using Import-Module

The `Import-Module` cmdlet allows us to add a module to the current PowerShell session.

All About Cmdlets and Modules

```
PS C:\Users\htb-student> Get-Help Import-Module
```

NAME

```
Import-Module
```

SYNOPSIS

```
Adds modules to the current session.
```

SYNTAX

```
Import-Module [-Assembly] <System.Reflection.Assembly[]> [-Alias <System.String[]>] [-ArgumentList <System.Object[]>] [-AsCustomObject] [-Cmdlet <System.String[]>] [-DisableNameChecking] [-Force] [-Function <System.String[]>] [-Global] [-NoClobber] [-PassThru] [-Prefix <System.String>] [-Scope {Local | Global}] [-Variable <System.String[]>] [<CommonParameters>]
```

```
Import-Module [-Name] <System.String[]> [-Alias <System.String[]>] [-ArgumentList <System.Object[]>] [-AsCustomObject] [-CimNamespace <System.String>] [-CimResourceUri <System.Uri>] [-CimSession <Microsoft.Management.Infrastructure.CimSession>] [-Cmdlet <System.String[]>] [-DisableNameChecking] [-Force] [-Function <System.String[]>] [-Global] [-MaximumVersion <System.String>] [-MinimumVersion <System.Version>] [-NoClobber] [-PassThru] [-Prefix <System.String>] [-RequiredVersion <System.Version>] [-Scope {Local | Global}] [-Variable <System.String[]>] [<CommonParameters>]
```

```
<SNIP>
```

To understand the idea of importing the module into our current PowerShell session, we can attempt to run a cmdlet (`Get-NetLocalgroup`) that is part of PowerSploit. We will get an error message when attempting to do this without importing a module.

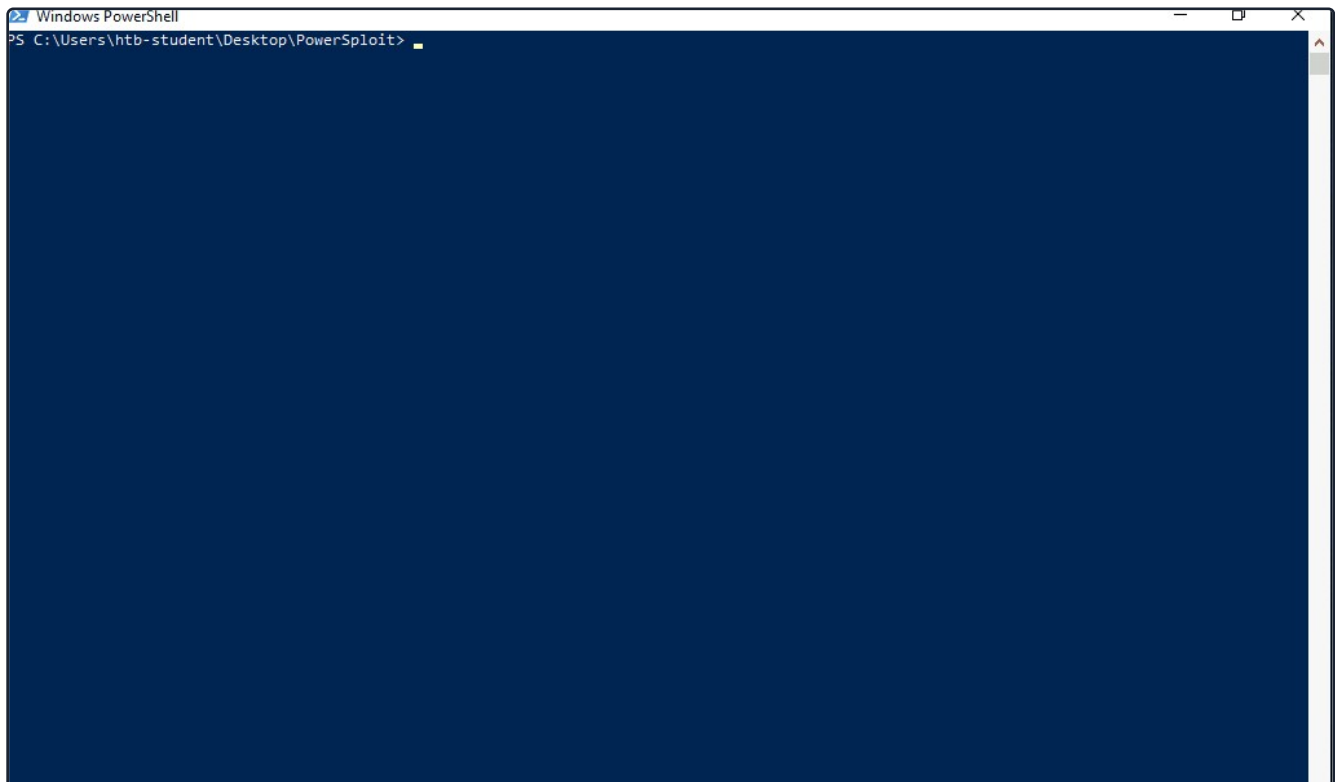
Once we successfully import the PowerSploit module (it has been placed on the target host's Desktop for our use), many cmdlets will be available to us, including `Get-NetLocalgroup`. See this in action in the clip below:

Importing PowerSploit.psd1

All About Cmdlets and Modules

```
PS C:\Users\htb-student\Desktop\PowerSploit> Import-Module .\PowerSploit.psd1
PS C:\Users\htb-student\Desktop\PowerSploit> Get-NetLocalgroup
```

ComputerName	GroupName	Comment
WS01	Access Control Assistance Operators	Members of this group can remotely query authorization attributes.
WS01	Administrators	Administrators have complete and unrestricted access to the computer.
WS01	Backup Operators	Backup Operators can override security restrictions for the system.
WS01	Cryptographic Operators	Members are authorized to perform cryptographic operations.
WS01	Distributed COM Users	Members are allowed to launch, activate and use Distributed COM objects.
WS01	Event Log Readers	Members of this group can read event logs from local machine.
WS01	Guests	Guests have the same access as members of the Users group by default.
WS01	Hyper-V Administrators	Members of this group have complete and unrestricted access to the system.
WS01	IIS_IUSRS	Built-in group used by Internet Information Services.
WS01	Network Configuration Operators	Members in this group can have some administrative privileges.
WS01	Performance Log Users	Members of this group may schedule logging of performance counter data.
WS01	Performance Monitor Users	Members of this group can access performance counter data locally.
WS01	Power Users	Power Users are included for backwards compatibility and possible future use.
WS01	Remote Desktop Users	Members in this group are granted the right to logon remotely.
WS01	Remote Management Users	Members of this group can access WMI resources over management.
WS01	Replicator	Supports file replication in a domain.
WS01	System Managed Accounts Group	Members of this group are managed by the system.
WS01	Users	Users are prevented from making accidental or intentional system changes.



Notice how at the beginning of the clip, `Get-NetLocalgroup` was not recognized. This event happened because it is not included in the default module path. We see where the default module path is by listing the environment variable `PSModulePath`.

Viewing PSModulePath

All About Cmdlets and Modules

```
PS C:\Users\htb-student> $env:PSModulePath
```

```
C:\Users\htb-student\Documents\WindowsPowerShell\Modules;C:\Program Files\WindowsPowerShell\Modules;C:\Windows\
```

When the `PowerSploit.psd1` module is imported, the `Get-NetLocalgroup` function is recognized. This happens because several

modules are included when we load Powersploit.psd1. It is possible to permanently add a module or several modules by adding the files to the referenced directories in the PSModulePath. This action makes sense if we were using a Windows OS as our primary attack host, but on an engagement, our time would be better off just transferring specific scripts over to the attack host and importing them as needed.

Execution Policy

An essential factor to consider when attempting to use PowerShell scripts and modules is [PowerShell's execution policy](#). As outlined in Microsoft's official documentation, an execution policy is not a security control. It is designed to give IT admins a tool to set parameters and safeguards for themselves.

Execution Policy's Impact

All About Cmdlets and Modules

```
PS C:\Users\htb-student\Desktop\PowerSploit> Import-Module .\PowerSploit.psd1

Import-Module : File C:\Users\Users\htb-student\PowerSploit.psm1
cannot be loaded because running scripts is disabled on this system. For more information, see
about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ Import-Module .\PowerSploit.psd1
+ ~~~~~
    + CategoryInfo          : SecurityError: (:) [Import-Module], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess,Microsoft.PowerShell.Commands.ImportModuleCommand
```

The host's execution policy makes it so that we cannot run our script. We can get around this, however. First, let us check our execution policy settings.

Checking Execution Policy State

All About Cmdlets and Modules

```
PS C:\htb> Get-ExecutionPolicy

Restricted
```

Our current setting restricts what the user can do. If we want to change the setting, we can do so with the [Set-ExecutionPolicy](#) cmdlet.

Setting Execution Policy

All About Cmdlets and Modules

```
PS C:\htb> Set-ExecutionPolicy undefined
```

By setting the policy to undefined, we are telling PowerShell that we do not wish to limit our interactions. Now we should be able to import and run our script.

Testing It Out

All About Cmdlets and Modules

```
PS C:\htb> Import-Module .\PowerSploit.psd1
```

```
Import-Module .\PowerSploit.psd1
PS C:\Users\htb> get-module
```

ModuleType	Version	Name	ExportedCommands
Manifest	3.1.0.0	Microsoft.PowerShell.Management	{Add-Computer, Add-Content, Check...
Manifest	3.0.0.0	Microsoft.PowerShell.Security	{ConvertFrom-SecureString, Conver...
Manifest	3.1.0.0	Microsoft.PowerShell.Utility	{Add-Member, Add-Type, Clear-Vari...
Script	3.0.0.0	PowerSploit	{Add-Persistence, Add-ServiceDacl...
Script	2.0.0	PSReadLine	{Get-PSReadLineKeyHandler, Get-PS...

Looking at our loaded modules, we can see that we successfully loaded PowerSploit. Now we can use the tools as needed.

Note: As a sysadmin, these kinds of changes are common and should always be reverted once we are done with work. As a pentester, us making a change like this and not reverting it could indicate to a defender that the host has been compromised. Be sure to check that we clean up after our actions. Another way we can bypass the execution policy and not leave a persistent change is to change it at the process level using -scope.

Change Execution Policy By Scope

All About Cmdlets and Modules

```
PS C:\htb> Set-ExecutionPolicy -scope Process
PS C:\htb> Get-ExecutionPolicy -list
```

Scope	ExecutionPolicy
MachinePolicy	Undefined
UserPolicy	Undefined
Process	Bypass
CurrentUser	Undefined
LocalMachine	Bypass

By changing it at the Process level, our change will revert once we close the PowerShell session. Keep the execution policy in mind when working with scripts and new modules. Of course, we want to look at the scripts we are trying to load first to ensure they are safe for use. As penetration testers, we may run into times when we need to be creative about how we bypass the Execution Policy on a host. This [blog post](#) has some creative ways that we have used on real-world engagements with great success.

Calling Cmdlets and Functions From Within a Module

If we wish to see what aliases, cmdlets, and functions an imported module brought to the session, we can use `Get-Command -Module <moduleName>` to enlighten us.

Using Get-Command

All About Cmdlets and Modules

```
PS C:\htb> Get-Command -Module PowerSploit
```

CommandType	Name	Version	Source
Alias	Invoke-ProcessHunter	3.0.0.0	PowerSploit
Alias	Invoke-ShareFinder	3.0.0.0	PowerSploit
Alias	Invoke-ThreadedFunction	3.0.0.0	PowerSploit
Alias	Invoke-UserHunter	3.0.0.0	PowerSploit
Alias	Request-SPNTicket	3.0.0.0	PowerSploit
Alias	Set-ADObject	3.0.0.0	PowerSploit
Function	Add-Persistence	3.0.0.0	PowerSploit
Function	Add-ServiceDacl	3.0.0.0	PowerSploit
Function	Find-AVSignature	3.0.0.0	PowerSploit
Function	Find-InterestingFile	3.0.0.0	PowerSploit

Function	Find-InterestingFile	3.0.0.0	Powersploit
Function	Find-LocalAdminAccess	3.0.0.0	PowerSploit
Function	Find-PathDLLHijack	3.0.0.0	PowerSploit
Function	Find-ProcessDLLHijack	3.0.0.0	PowerSploit
Function	Get-ApplicationHost	3.0.0.0	PowerSploit
Function	Get-GPPPassword	3.0.0.0	PowerSploit

Now we can see what was loaded by PowerSploit. From this point, we can use the scripts and functions as needed. This is the easy part, pick the function and let it run.

Deep Dive: Finding & Installing Modules from PowerShell Gallery & GitHub

In today's day and age, sharing information is extremely easy. That goes for solutions and new creations as well. When it comes to PowerShell modules, the [PowerShell Gallery](#) is the best place for that. It is a repository that contains PowerShell scripts, modules, and more created by Microsoft and other users. They can range from anything as simple as dealing with user attributes to solving complex cloud storage issues.

PowerShell Gallery

PowerShell Gallery Packages Publish Statistics Documentation Sign in

Welcome to the PowerShell Gallery

The central repository for sharing and acquiring PowerShell code including PowerShell modules, scripts, and DSC resources.

Search PowerShell packages:

Az, etc...

10,635 Unique Packages 6,155,965,410 Total package downloads 153,410 Total packages

Learn

[What is PowerShell Gallery?](#)
Learn why the PowerShell Gallery is the most used resource for sharing and acquiring PowerShell code.

[Getting started](#)
Learn how to install and set up the PowerShellGet module, which is required in order to download packages from the Gallery.

Top package downloads (last 6 weeks):

Package	Downloads
NetworkingDsc	6,088,672
PSWindowsUpdate	3,740,842
Az.Accounts	3,509,582
PackageManagement	2,967,790
ComputerManagementDsc	2,903,043
Az.Storage	2,863,571

[View more](#)

Conveniently for us, There is already a module built into PowerShell meant to help us interact with the PowerShell Gallery called [PowerShellGet](#). Let us take a look at it:

PowerShellGet

All About Cmdlets and Modules			
PS C:\htb> Get-Command -Module PowerShellGet			
CommandType	Name	Version	Source
-----	----	-----	-----
Function	Find-Command	1.0.0.1	PowerShellGet
Function	Find-DscResource	1.0.0.1	PowerShellGet
Function	Find-Module	1.0.0.1	PowerShellGet
Function	Find-RoleCapability	1.0.0.1	PowerShellGet

Function	Find-Script	1.0.0.1	PowerShellGet
Function	Get-InstalledModule	1.0.0.1	PowerShellGet
Function	Get-InstalledScript	1.0.0.1	PowerShellGet
Function	Get-PSRepository	1.0.0.1	PowerShellGet
Function	Install-Module	1.0.0.1	PowerShellGet
Function	Install-Script	1.0.0.1	PowerShellGet
Function	New-ScriptFileInfo	1.0.0.1	PowerShellGet
Function	Publish-Module	1.0.0.1	PowerShellGet
Function	Publish-Script	1.0.0.1	PowerShellGet
Function	Register-PSRepository	1.0.0.1	PowerShellGet
Function	Save-Module	1.0.0.1	PowerShellGet
Function	Save-Script	1.0.0.1	PowerShellGet
Function	Set-PSRepository	1.0.0.1	PowerShellGet
Function	Test-ScriptFileInfo	1.0.0.1	PowerShellGet
Function	Uninstall-Module	1.0.0.1	PowerShellGet
Function	Uninstall-Script	1.0.0.1	PowerShellGet
Function	Unregister-PSRepository	1.0.0.1	PowerShellGet
Function	Update-Module	1.0.0.1	PowerShellGet
Function	Update-ModuleManifest	1.0.0.1	PowerShellGet
Function	Update-Script	1.0.0.1	PowerShellGet
Function	Update-ScriptFileInfo	1.0.0.1	PowerShellGet

This module has many different functions to help us work with and download existing modules from the gallery and make and upload our own. From our function listing, let us give Find-Module a try. One module that will prove extremely useful to system admins is the [AdminToolbox](#) module. It is a collection of several other modules with tools meant for Active Directory management, Microsoft Exchange, virtualization, and many other tasks an admin would need on any given day.

Find-Module

All About Cmdlets and Modules			
PS C:\htb> Find-Module -Name AdminToolbox			
Version	Name	Repository	Description
-----	----	-----	-----
11.0.8	AdminToolbox	PSGallery	Master module for a col...

Like with many other PowerShell cmdlets, we can also search using wildcards. Once we have found a module we wish to utilize, installing it is as easy as [Install-Module](#). Remember that it requires administrative rights to install modules in this manner.

Install-Module

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/powershell

PS C:\Windows\system32> find-module -Name AdminToolbox | Install-module
```

In the image above, we chained [Find-Module](#) with [Install-Module](#) to simultaneously perform both actions. This example takes

In the image above, we chained `Find-Module` with `Install-Module` to simultaneously perform both actions. This example takes advantage of PowerShell's Pipeline functionality. We will cover this deeper in another section, but for now, it allowed us to find and install the module with one command string. Remember that modern instances of PowerShell will auto-import a module installed the first time we run a cmdlet or function from it, so there is no need to import the module after installing it. This differs from custom modules or modules we bring onto the host (from GitHub, for example). We will have to manually import it each time we want to use it unless we modify our PowerShell Profile. We can find the locations for each specific PowerShell profile [Here](#). Besides creating our own modules and scripts or importing them from the PowerShell Gallery, we can also take advantage of [Github](#) and all the amazing content the IT community has come up with externally. Utilizing [Git](#) and [Github](#) for now requires the installation of other applications and knowledge of other concepts we have yet to cover, so we will save this for later in the module.

Tools To Be Aware Of

Below we will quickly list a few PowerShell modules and projects we, as penetration testers and sysadmins, should be aware of. Each of these tools brings a new capability to use within PowerShell. Of course, there are plenty more than just our list; these are just several we find ourselves returning to on every engagement.

- [AdminToolbox](#): AdminToolbox is a collection of helpful modules that allow system administrators to perform any number of actions dealing with things like Active Directory, Exchange, Network management, file and storage issues, and more.
- [ActiveDirectory](#): This module is a collection of local and remote administration tools for all things Active Directory. We can manage users, groups, permissions, and much more with it.
- [Empire / Situational Awareness](#): Is a collection of PowerShell modules and scripts that can provide us with situational awareness on a host and the domain they are apart of. This project is being maintained by [BC Security](#) as a part of their Empire Framework.
- [Inveigh](#): Inveigh is a tool built to perform network spoofing and Man-in-the-middle attacks.
- [BloodHound / SharpHound](#): Bloodhound/SharpHound allows us to visually map out an Active Directory Environment using graphical analysis tools and data collectors written in C# and PowerShell.

Working with PowerShell modules and cmdlets is intuitive and easy to master quickly. This skill will come in handy for the rest of this module since we will be dealing with various tools and topics within PowerShell that may require us to install, import, or examine modules and cmdlets. If you get stuck, be sure to refer back to this section. Now it is time to move on to User and Group management.

VPN Servers

⚠ Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

● UDP 1337 ● TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

58ms ▼

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)



Cheat Sheet

Download VPN
Connection File

+ 0

What cmdlet can help us find modules that are loaded into our session?

Submit your answer here...



Submit



Hint

+ 0

What module provides us with cmdlets built to manage package installation from the PowerShell Gallery?

Submit your answer here...



Submit



SSH to with user "htb-student" and password "HTB_@cademy_stdnt!"

+ 0

Take a moment to practice installing and loading modules on the target host. Answer "COMPLETE" when done.

Submit your answer here...

Submit

Hint

← Previous

Next →

Cheat Sheet

Resources

? Go to Questions

Table of Contents

Introduction



CMD

Command Prompt Basics



Getting Help



System Navigation



Working with Directories and Files



Gathering System Information



Finding Files and Directories



Environment Variables



Managing Services



Working With Scheduled Tasks



PowerShell

CMD Vs. PowerShell



All About Cmdlets and Modules

User and Group Management

Working with Files and Directories

Finding & Filtering Content

Working with Services

Working with the Registry

Working with the Windows Event Log

Networking Management from The CLI

Interacting with The Web

PowerShell Scripting and Automation

Finish Strong

Skills Assessment

Beyond This Module

My Workstation

O F F L I N E

▶ Start Instance

∞ / 1 spawns left